

PROJECT REPORT

NAME = ARCHITA SINGH

REG NO = 25BAI11553

COURSE = PROGRAMMING IN JAVA

1. Introduction

Student Course & Academic Management System is a Java command line application that was developed to manage student related academic data in an organized manner. The application helps the user in storing and managing the student's information, course details, grades along with their enrollment records.

The software is capable of calculating the grade points of the grades and with the help of credit hours allocates the final GPA of the student. All the records are stored in the text files by the application through Java file handling.

The project showcases the use of classes and objects, encapsulation, ArrayList, methods, conditional statements, loops, and exception handling along with the input validation and file handling concepts of Java.

2. Problem Statement

Maintaining the student's data manually in an organized manner is a challenging task, including their courses, information about their registration, and grades. It becomes highly time-consuming to perform operations like searching for a particular record or calculating the average grade of a student.

The Student Course & Academic Management System is a java command line application that aims to offer an easy solution to manage students' basic academic information like grades and marks.

The application not only stores the grades of the student but also converts the grades into grade points and finally calculates the weighted average grade point as per the credit system of the respective course. It presents a complete academic report of the student with GPA.

All the records are stored in the text files using java file handling concepts.

The project demonstrates the implementation of Object-oriented programming, classes and objects, encapsulation, ArrayList, methods, input validation, exception handling, and file handling concepts in java.

3.Functional Requirements

The system should provide the following set of functional requirements.

FR1 – Student Management

Add a student's data: ID, name, email, and phone.

View student data.

Search for a student by Student ID.

FR2 – Course Management

Add a course: course ID, course name, and credits.

View course data.

Search for a course by Course ID.

FR3 – Enrollment Management

Enroll a student in a course.

View enrollment data.

FR4 – Grade Management

Add a grade for every course.

Convert a grade into grade points according to the grade point system.

Calculate weighted grade points for every course based on the credit hours.

FR5 – Academic Report

Generate a student's academic report.

View the student's course, grade, credit hours, and grade points.

Compute the student's GPA.

FR6 – File Storage

Store student data in the students.txt file.

Store course data in the courses.txt file.

Store student enrollment data in the enrollment.txt file.

Store student's grade data in the grades.txt file.

5.Non-Functional Requirements

NFR1 – Usability

The application menus should use simple command prompt prompts and make it easy for a user to enter a command.

NFR2 – Reliability

The application should not crash when receiving incorrect input.

NFR3 – Maintainability

The application's source code should be split into class definitions: Student, Course, Grade, and manager classes. This will make the code easy to read and maintain.

NFR4 – Error Handling

The application should be able to process numeric user input. It should also handle file reading/writing exceptions using C++ built-in exception handling mechanism.

NFR5 – Storage

Student records, course catalog, enrollment data, and student grades should be saved in separate text files upon exit or during application execution to persist data between application sessions.

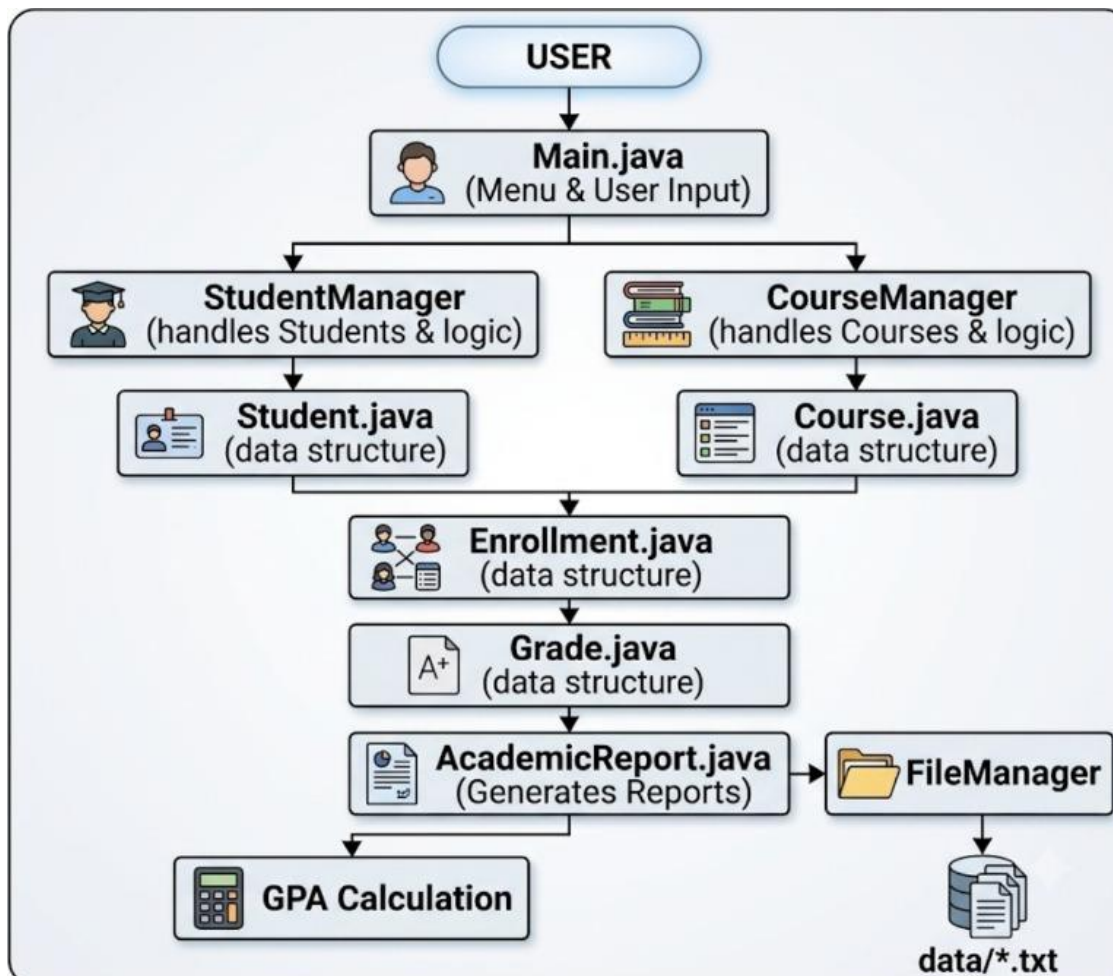
NFR6 – Performance

The application should handle student and academic record management for a small scale efficiently.

5) DESIGN DIAGRAM

System Architecture

The project follows a **modular object-oriented architecture**. The `Main.java` class controls the overall program flow and interacts with manager classes, entity classes, the academic report class, and the file storage component.



6. Application Workflow

Student Details



Student Management



Course Details



Course Management



Enrollment



Grade Entry



Academic Report



GPA Calculation



File Storage

Architecture Explanation

Main.java – controls the general execution and collects the inputs from the user.

Student.java – represents the data of student.

StudentManager.java – manages the student objects and provides search of students.

Course.java – represents the course data.

CourseManager.java – manages the courses and provides searching of courses.

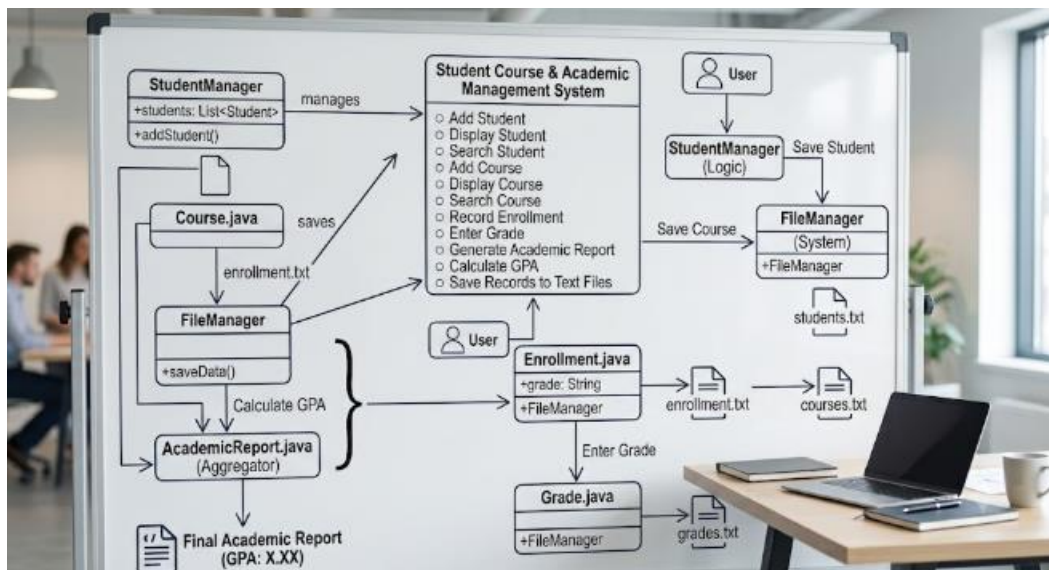
Enrollment.java – relates the student and the course.

Grade.java – retains the grades and computes the grade points.

AcademicReport.java – prepares the academic report and calculates the GPA.

FileManager.java – stores the records in the text files.

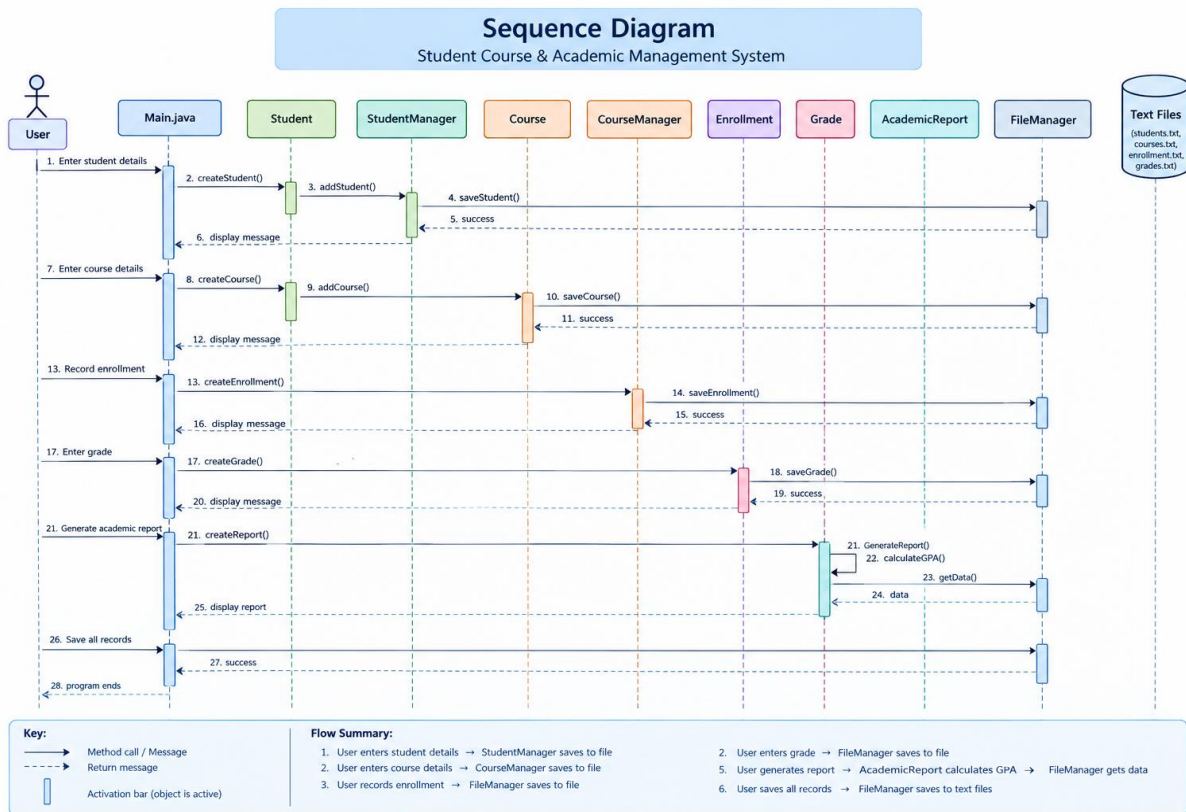
2. Use Case Diagram



3. Class Diagram



Sequence Diagram



Project Structure

The project is structured into separate Java classes according to their responsibilities. The folder with the name execution accommodates the java files with sources of the code, and the data folder was used for the text files with the records. Screenshots of the working application were separated in a different folder.

student management system

execution

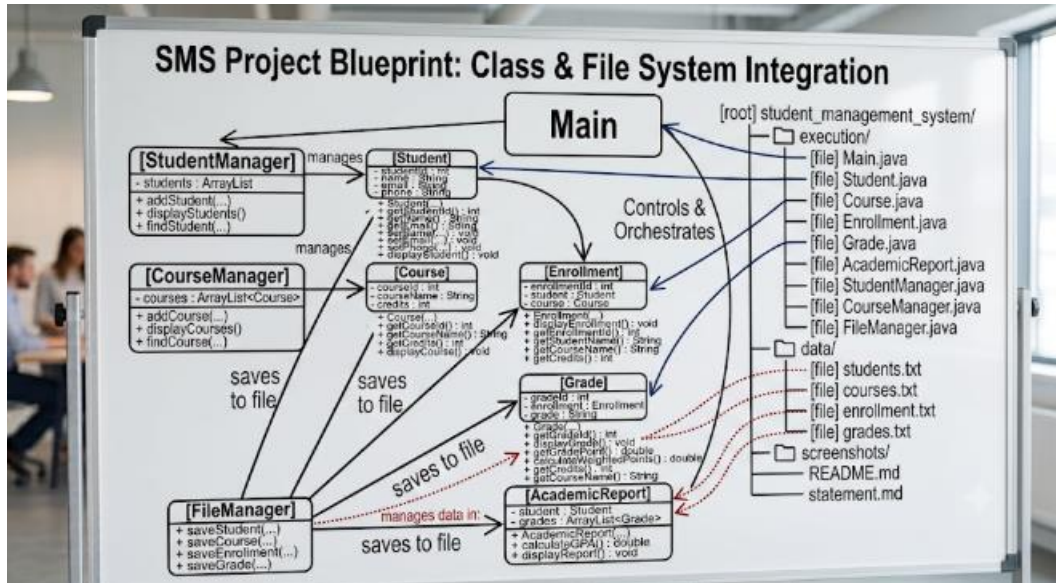
- Main.java
- Student.java
- Course.java
- Enrollment.java
- Grade.java
- AcademicReport.java
- StudentManager.java
- CourseManager.java
- FileManager.java

data

- students.txt
- courses.txt
- enrollment.txt
- grades.txt

screenshots

Statement.md



4. Implementation

8.1 Student Management

The Student class holds the student's ID, name, email, and phone number while the StudentManager stores student objects and offers the capability to view and search for the student.

8.2 Course Management

We have the Course class that holds the course ID, course name, and credits. Similarly, the Course Manager is tasked with managing course objects, displaying, and searching for a course using the course ID.

8.3 Enrollment Management

The Enrollment class holds the student's data, the course they have enrolled, and the unique enrollment ID. The class offers a feature to display the enrollment's details.

8.4 Grade Management

The Grade class stores the grade of the student's enrollment. The grades are in letter form, for example., A, B, C, D, and E. Besides storing, the class converts the grades into grade points ranging from 4 to 0. We have calculated the weighted points using the formula below.

$$\text{Weighted Points} = \text{Grade Point} \times \text{Course Credits}$$

8.5 Academic Report

The AcademicReport class takes a student's grade and credits to generate their grades report. In this class, we have implemented the GPA calculation formula as follows:

$$\text{GPA} = \text{Total Weighted Points} \div \text{Total Credits}$$

8.6 File Handling

The file handling module is managed by the File Manager. It uses the Java FileWriter to write information into files. The files are stored in text format.

The files include:

students.txt

courses.txt

enrollment.txt

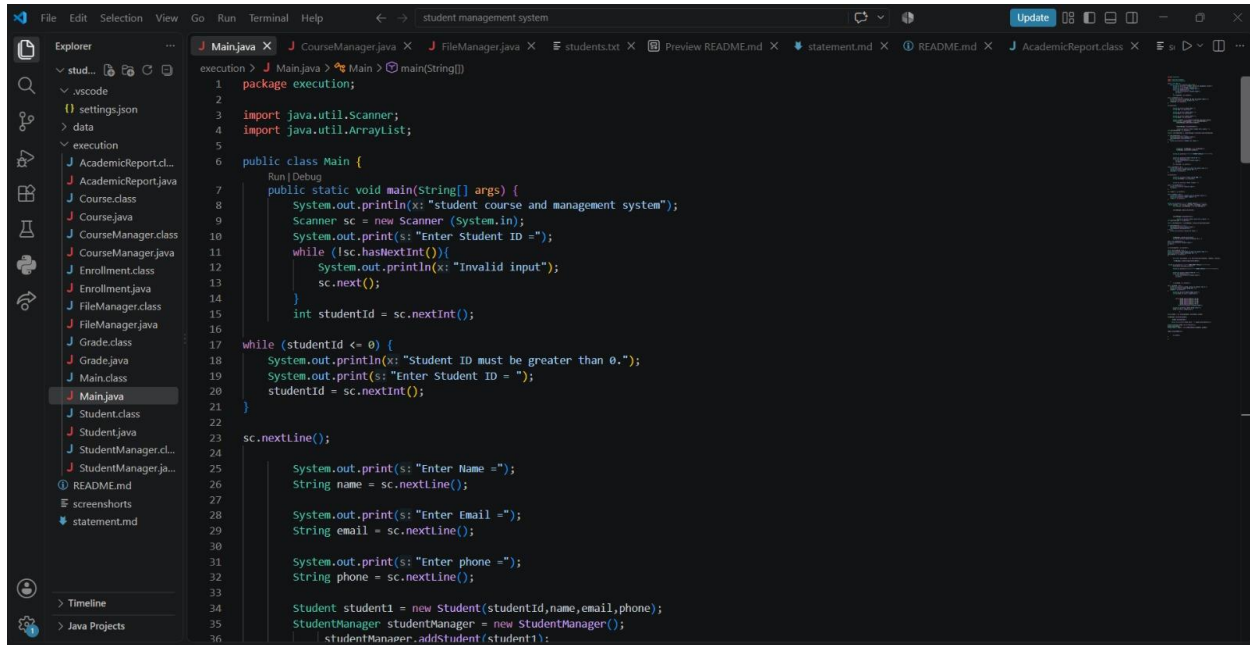
grades.txt

8.7 Input Validation

Finally, the Main class offers the input validation feature for IDs, credits, and grades. When invalid input is detected, the application throws an error. For numeric values, the application only allows decimal numbers. On the other hand, grades are only allowed between A-E.

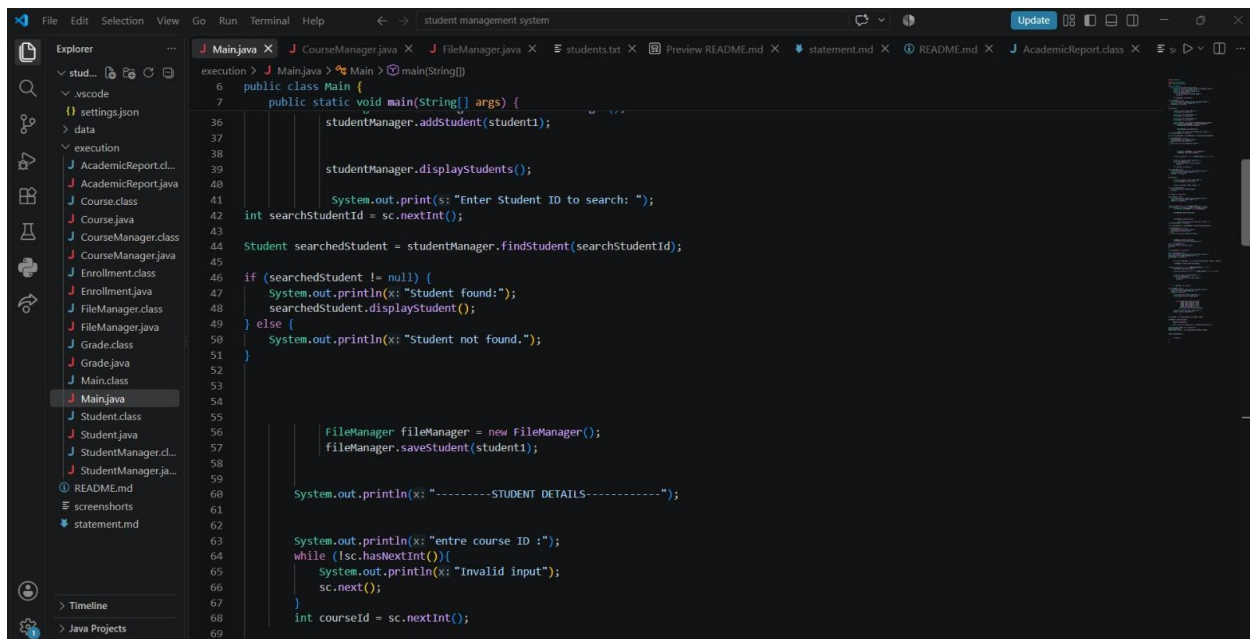
Screenshots & Results

Code- Main.java



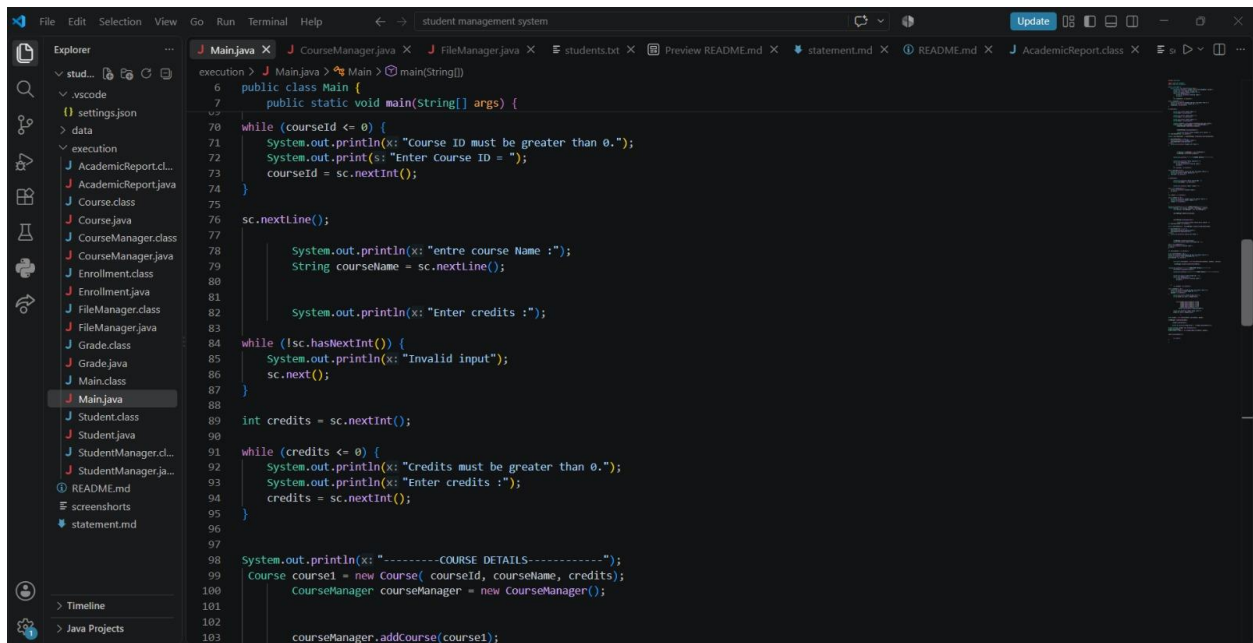
The screenshot shows the VS Code editor with the 'Main.java' file open. The Explorer sidebar on the left lists various files and folders, including 'Main.java' which is currently selected. The main editor area displays the following code:

```
1 package execution;
2
3 import java.util.Scanner;
4 import java.util.ArrayList;
5
6 public class Main {
7     public static void main(String[] args) {
8         System.out.println(x: "student course and management system");
9         Scanner sc = new Scanner (system.in);
10        System.out.print(x: "Enter Student ID = ");
11        while (!sc.hasNextInt()){
12            System.out.println(x: "Invalid input");
13            sc.next();
14        }
15        int studentId = sc.nextInt();
16
17        while (studentId <= 0) {
18            System.out.println(x: "Student ID must be greater than 0.");
19            System.out.print(s: "Enter Student ID = ");
20            studentId = sc.nextInt();
21        }
22
23        sc.nextLine();
24
25        System.out.print(s: "Enter Name = ");
26        String name = sc.nextLine();
27
28        System.out.print(s: "Enter Email = ");
29        String email = sc.nextLine();
30
31        System.out.print(s: "Enter phone = ");
32        String phone = sc.nextLine();
33
34        Student student1 = new Student(studentId,name,email,phone);
35        StudentManager studentManager = new StudentManager();
36        studentManager.addStudent(student1);
```

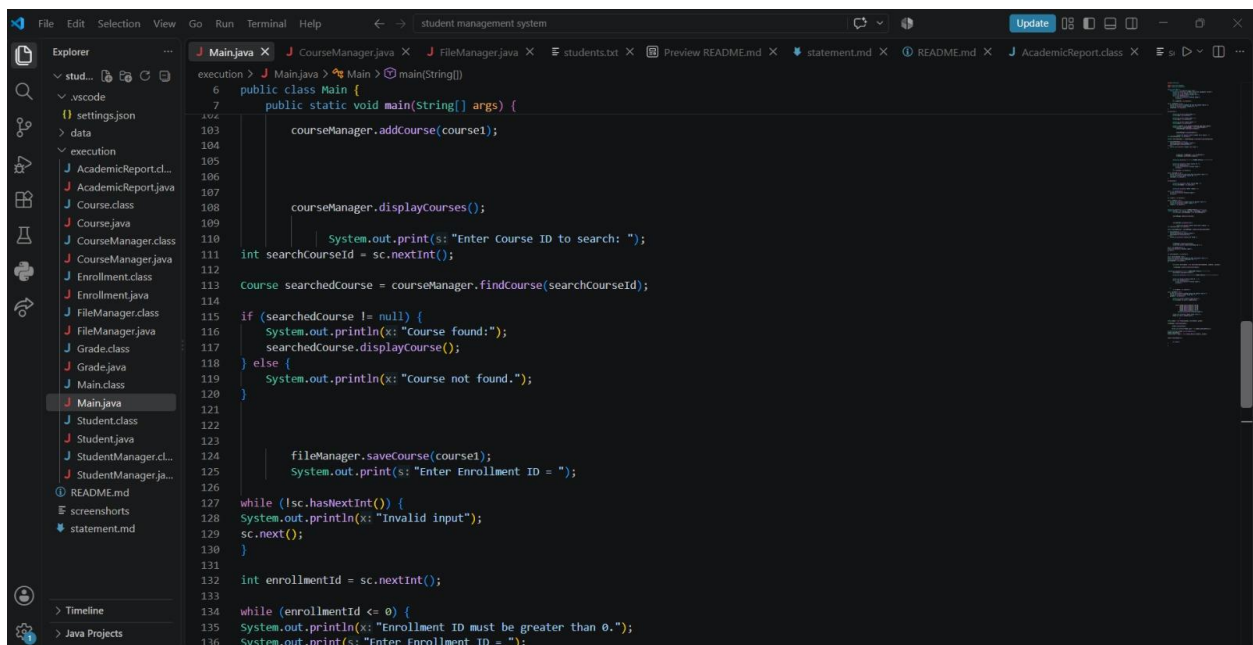


The screenshot shows the VS Code editor with the 'Main.java' file open, displaying the continuation of the code from the previous block:

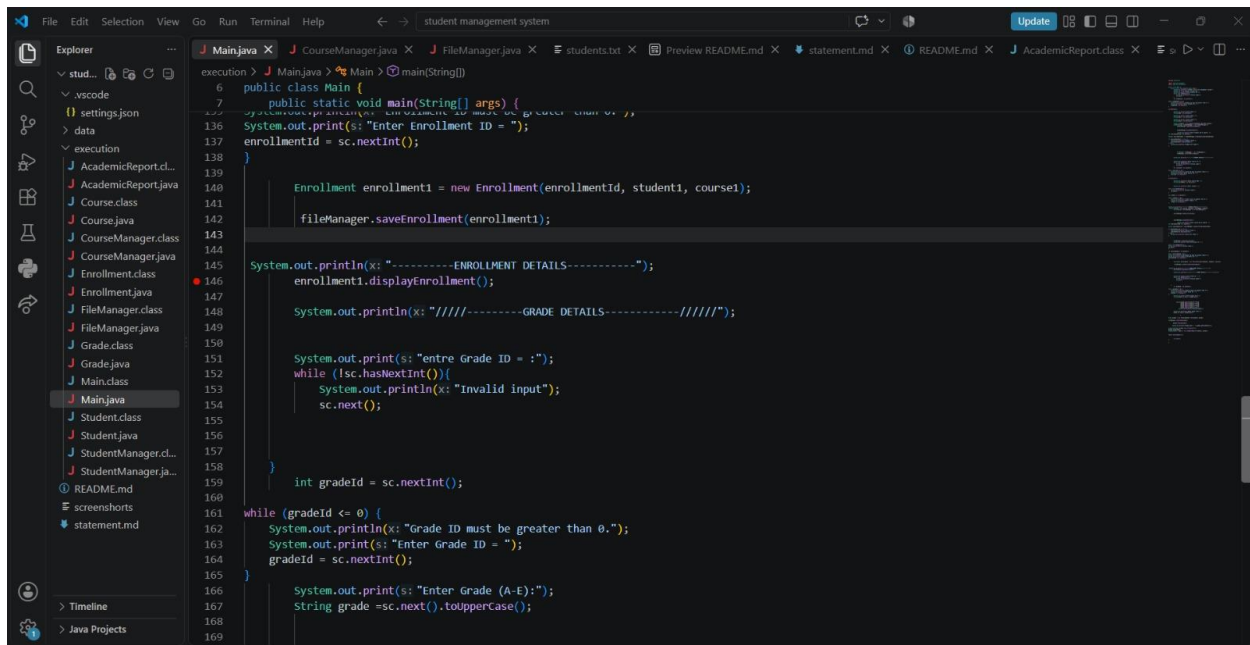
```
36        studentManager.addStudent(student1);
37
38        studentManager.displayStudents();
39
40        System.out.print(s: "Enter Student ID to search: ");
41        int searchStudentId = sc.nextInt();
42
43        Student searchedStudent = studentManager.findStudent(searchStudentId);
44
45        if (searchedStudent != null) {
46            System.out.println(x: "Student found:");
47            searchedStudent.displayStudent();
48        } else {
49            System.out.println(x: "Student not found.");
50        }
51
52
53
54
55        FileManager fileManager = new FileManager();
56        fileManager.saveStudent(student1);
57
58        System.out.println(x: "-----STUDENT DETAILS-----");
59
60
61        System.out.println(x: "entre course ID :");
62        while (!sc.hasNextInt()){
63            System.out.println(x: "Invalid input");
64            sc.next();
65        }
66        int courseId = sc.nextInt();
67
68
69
```



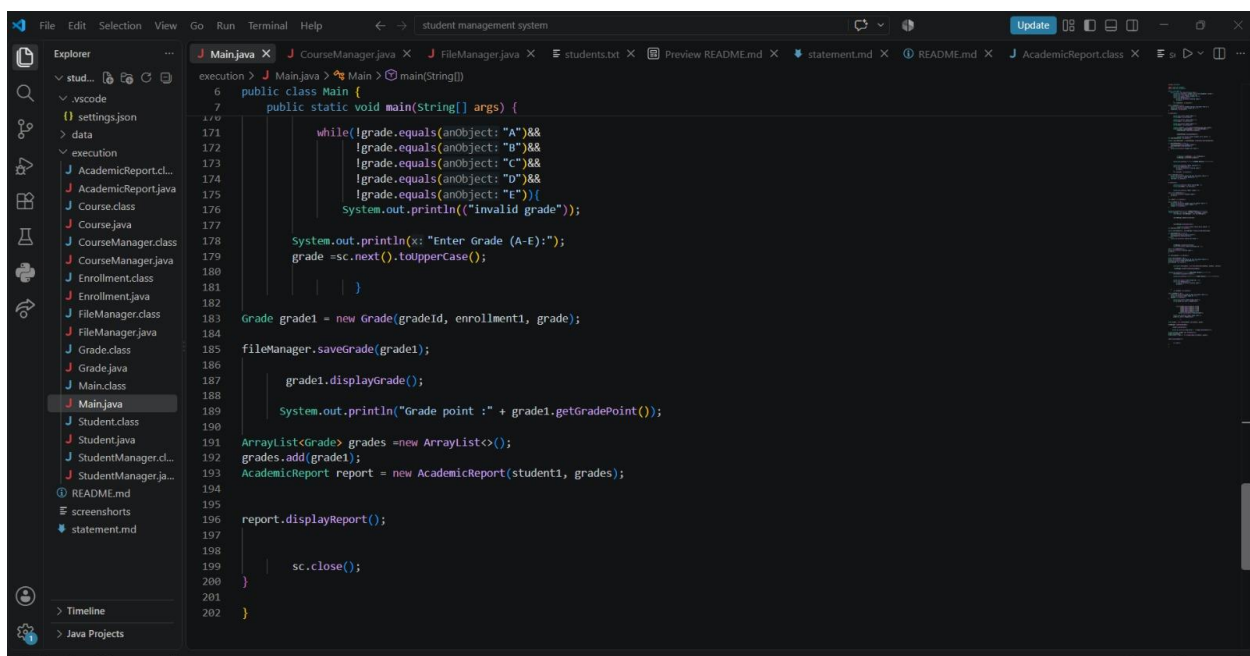
```
6 public class Main {
7     public static void main(String[] args) {
8
9         while (courseId <= 0) {
10             System.out.println(x: "Course ID must be greater than 0.");
11             System.out.print(s: "Enter Course ID = ");
12             courseId = sc.nextInt();
13         }
14
15         sc.nextLine();
16
17         System.out.println(x: "entre course Name :");
18         String courseName = sc.nextLine();
19
20         System.out.println(x: "Enter credits :");
21
22         while (!sc.hasNextInt()) {
23             System.out.println(x: "Invalid input");
24             sc.next();
25         }
26
27         int credits = sc.nextInt();
28
29         while (credits <= 0) {
30             System.out.println(x: "Credits must be greater than 0.");
31             System.out.println(x: "Enter credits :");
32             credits = sc.nextInt();
33         }
34
35         System.out.println(x: "-----COURSE DETAILS-----");
36         Course course1 = new Course(courseId, courseName, credits);
37         CourseManager courseManager = new CourseManager();
38
39         courseManager.addCourse(course1);
40     }
41 }
```



```
103         courseManager.addCourse(course1);
104
105         courseManager.displayCourses();
106
107         System.out.print(s: "Enter Course ID to search: ");
108         int searchCourseId = sc.nextInt();
109
110         Course searchedCourse = courseManager.findCourse(searchCourseId);
111
112         if (searchedCourse != null) {
113             System.out.println(x: "Course found:");
114             searchedCourse.displaycourse();
115         } else {
116             System.out.println(x: "Course not found.");
117         }
118
119         FileManager.saveCourse(course1);
120         System.out.print(s: "Enter Enrollment ID = ");
121
122         while (!sc.hasNextInt()) {
123             System.out.println(x: "Invalid input");
124             sc.next();
125         }
126
127         int enrollmentId = sc.nextInt();
128
129         while (enrollmentId <= 0) {
130             System.out.println(x: "Enrollment ID must be greater than 0.");
131             System.out.print(s: "Enter enrollment ID = ");
132         }
133     }
134 }
```



```
1  public class Main {
2      public static void main(String[] args) {
3          Scanner scanner = new Scanner(System.in);
4          System.out.print(s: "Enter Enrollment ID = ");
5          enrollmentId = scanner.nextInt();
6      }
7
8      Enrollment enrollment1 = new Enrollment(enrollmentId, student1, course1);
9      fileManager.saveEnrollment(enrollment1);
10
11      System.out.println(x: "-----ENROLLMENT DETAILS-----");
12      enrollment1.displayEnrollment();
13
14      System.out.println(x: "/////-----GRADE DETAILS-----\\\\\\\\\\\\\\\\");
15
16      System.out.print(s: "Enter Grade ID = ");
17      while (!scanner.hasNextInt()){
18          System.out.println(x: "Invalid input");
19          scanner.next();
20      }
21      int gradeId = scanner.nextInt();
22
23      while (gradeId <= 0) {
24          System.out.println(x: "Grade ID must be greater than 0.");
25          System.out.print(s: "Enter Grade ID = ");
26          gradeId = scanner.nextInt();
27      }
28
29      System.out.print(s: "Enter Grade (A-E):");
30      String grade = scanner.next().toUpperCase();
31  }
```



```
1  public class Main {
2      public static void main(String[] args) {
3          Scanner scanner = new Scanner(System.in);
4          System.out.print(s: "Enter Enrollment ID = ");
5          enrollmentId = scanner.nextInt();
6      }
7
8      Enrollment enrollment1 = new Enrollment(enrollmentId, student1, course1);
9      fileManager.saveEnrollment(enrollment1);
10
11      System.out.println(x: "-----ENROLLMENT DETAILS-----");
12      enrollment1.displayEnrollment();
13
14      System.out.println(x: "/////-----GRADE DETAILS-----\\\\\\\\\\\\\\\\");
15
16      System.out.print(s: "Enter Grade ID = ");
17      while (!scanner.hasNextInt()){
18          System.out.println(x: "Invalid input");
19          scanner.next();
20      }
21      int gradeId = scanner.nextInt();
22
23      while (gradeId <= 0) {
24          System.out.println(x: "Grade ID must be greater than 0.");
25          System.out.print(s: "Enter Grade ID = ");
26          gradeId = scanner.nextInt();
27      }
28
29      System.out.print(s: "Enter Grade (A-E):");
30      String grade = scanner.next().toUpperCase();
31
32      Grade grade1 = new Grade(gradeId, enrollment1, grade);
33      fileManager.saveGrade(grade1);
34      grade1.displayGrade();
35      System.out.println("Grade point : " + grade1.getGradePoint());
36
37      ArrayList<Grade> grades = new ArrayList<>();
38      grades.add(grade1);
39      AcademicReport report = new AcademicReport(student1, grades);
40      report.displayReport();
41      scanner.close();
42  }
```

Output screenshot

```

PS C:\Users\archita singh\OneDrive\Desktop\student management sy
stem>
PS C:\Users\archita singh\OneDrive\Desktop\student management sy
stem> javac execution\Main.java execution\Student.java execution
\Course.java execution\Enrollment.java execution\Grade.java exec
ution\AcademicReport.java execution\StudentManager.java executio
n\CourseManager.java execution\FileManager.java
PS C:\Users\archita singh\OneDrive\Desktop\student management sy
stem> java execution.Main
student course and management system
Enter Student ID =123
Enter Name =AYUSH
Enter Email =ayush@gmail.com
Enter phone =123456789
Student ID : 123
Name : AYUSH
Email : ayush@gmail.com
Phone : 123456789
-----
Enter Student ID to search: 123
Student found:
Student ID : 123
Name : AYUSH
Email : ayush@gmail.com
Phone : 123456789
-----STUDENT DETAILS-----
entre course ID :
1003
entre course Name :
java
Enter credits :
2
-----COURSE DETAILS-----
Course ID =1003
Course Name =java
Credits =2
-----
Enter Course ID to search: 1200
Course not found.

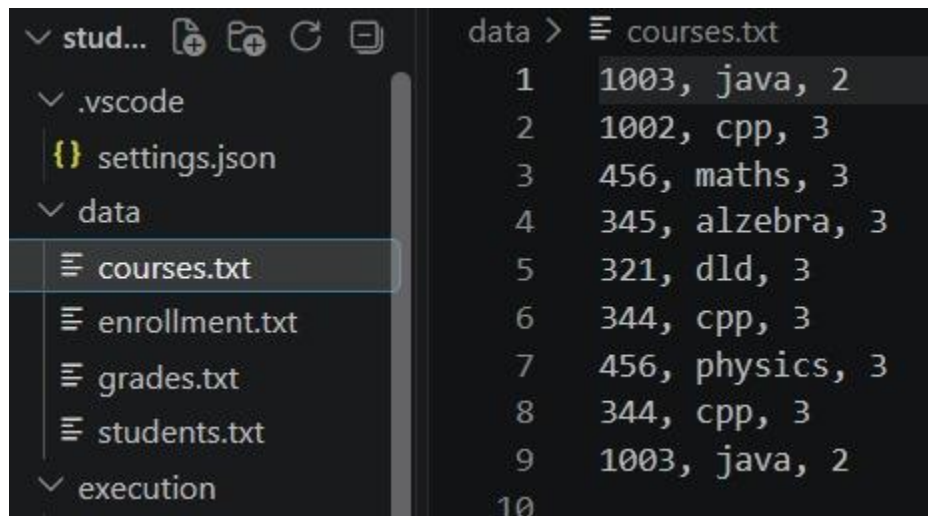
```

```
-----COURSE DETAILS-----
Course ID =1003
Course Name =java
Credits =2
-----
Enter Course ID to search: 1200
Course not found.
Enter Enrollment ID = 3411
-----ENROLLMENT DETAILS-----
Enrollment ID :3411
Student : AYUSH
Course :java
/////-----GRADE DETAILS-----/////
entre Grade ID = :556
Enter Grade (A-E):A
Grade Id :556
student:AYUSH
course :java
grade :A
Grade point :10.0

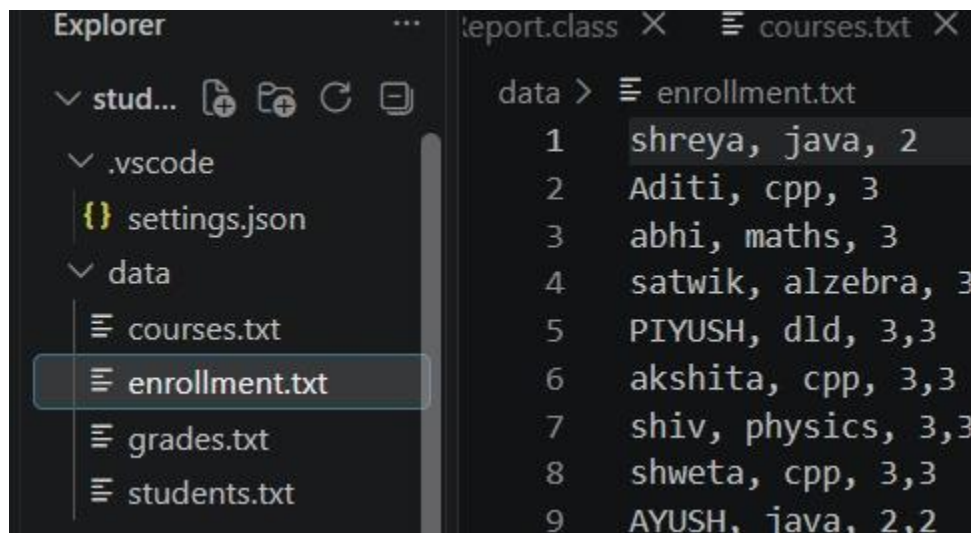
-----ACADEMIC REPORT-----
student ID :123
Name :AYUSH

Course :java
Grade Point: 10.0
Credits : 2
-----
final GPA :10.0
-----
PS C:\Users\archita singh\OneDrive\Desktop\student management sy
stem> 
```

Data storage screenshot



```
data > courses.txt
1 1003, java, 2
2 1002, cpp, 3
3 456, maths, 3
4 345, alzebra, 3
5 321, dld, 3
6 344, cpp, 3
7 456, physics, 3
8 344, cpp, 3
9 1003, java, 2
10
```



```
Explorer
data > enrollment.txt
1 shreya, java, 2
2 Aditi, cpp, 3
3 abhi, maths, 3
4 satwik, alzebra, 3
5 PIYUSH, dld, 3,3
6 akshita, cpp, 3,3
7 shiv, physics, 3,3
8 shweta, cpp, 3,3
9 AYUSH, java, 2,2
```

The screenshot shows the VS Code interface. The Explorer sidebar on the left displays a file tree with folders 'stud...' and '.vscode'. Under 'stud...', there is a 'data' folder containing 'courses.txt', 'enrollment.txt', 'grades.txt' (highlighted), and 'students.txt'. The Editor pane on the right shows the 'grades.txt' file with the following content:

```
data > grades.txt
1 java, 2, 6.0
2 cpp, 3, 8.0
3 maths, 3, 10.0
4 671,alzebra, 3, 6.0
5 788,dld, 3, 10.0
6 123,cpp, 3, 10.0
7 356,physics, 3, 8.0
8 456,cpp, 3, 5.0
9 556,java, 2, 10.0
```

The screenshot shows the VS Code interface. The Explorer sidebar on the left displays a file tree with folders 'stud...' and '.vscode'. Under 'stud...', there is a 'data' folder containing 'courses.txt', 'enrollment.txt', 'grades.txt', and 'students.txt' (highlighted). The Editor pane on the right shows the 'students.txt' file with the following content:

```
data > students.txt
1 102, shreya, shreya@gmail.com, 123456789
2 123, Aditi, aditi@gmail.com, 123456789
3 134, abhi, abhi@gmail.com, 156789123
4 145, satwik, satwik@gmail.com, 123456789
5 12, PIYUSH, Piyush@gmail.com, 56789234
6 123, akshita, akshita@gmail.com, 123456789
7 12, shiv, shiv@gmail.com, 123456789
8 567, shweta, shweta@gmail.com, 123456789
9 123, AYUSH, ayush@gmail.com, 123456789
```

Problem Statement

Maintaining the student's data manually in an organized manner is a challenging task, including their courses, information about their registration, and grades. It becomes highly time-consuming to perform operations like searching for a particular record or calculating the average grade of a student.

The Student Course & Academic Management System is a java command line application that aims to offer an easy solution to manage students' basic academic information like grades and marks.

The application not only stores the grades of the student but also converts the grades into grade points and finally calculates the weighted average grade point as per the credit system of the respective course. It presents a complete academic report of the student with GPA.

All the records are stored in the text files using java file handling concepts.

The project demonstrates the implementation of Object-oriented programming, classes and objects, encapsulation, ArrayList, methods, input validation, exception handling, and file handling concepts in java.

2. Objectives

The project has several objectives. They include:

Implementation of a student information system.

Course and credit management.

A record of student enrollments.

A record of student grades.

Converting grades into grade points.

Calculating weighted grade points using credit hours.

Generation of an academic report that contains the GPA.

Storage of records in text files.

Application of object-oriented programming concepts in a Java application.

3. Functional Requirements

The system should provide the following set of functional requirements.

FR1 – Student Management

Add a student's data: ID, name, email, and phone.

View student data.

Search for a student by Student ID.

FR2 – Course Management

Add a course: course ID, course name, and credits.

View course data.

Search for a course by Course ID.

FR3 – Enrollment Management

Enroll a student in a course.

View enrollment data.

FR4 – Grade Management

Add a grade for every course.

Convert a grade into grade points according to the grade point system.

Calculate weighted grade points for every course based on the credit hours.

FR5 – Academic Report

Generate a student's academic report.

View the student's course, grade, credit hours, and grade points.

Compute the student's GPA.

FR6 – File Storage

Store student data in the students.txt file.

Store course data in the courses.txt file.

Store student enrollment data in the enrollment.txt file.

Store student's grade data in the grades.txt file.

4. Non-Functional Requirements

NFR1 – Usability

The application menus should use simple command prompt prompts and make it easy for a user to enter a command.

NFR2 – Reliability

The application should not crash when receiving incorrect input.

NFR3 – Maintainability

The application's source code should be split into class definitions: Student, Course, Grade, and manager classes. This will make the code easy to read and maintain.

NFR4 – Error Handling

The application should be able to process numeric user input. It should also handle file reading/writing exceptions using C++ built-in exception handling mechanism.

NFR5 – Storage

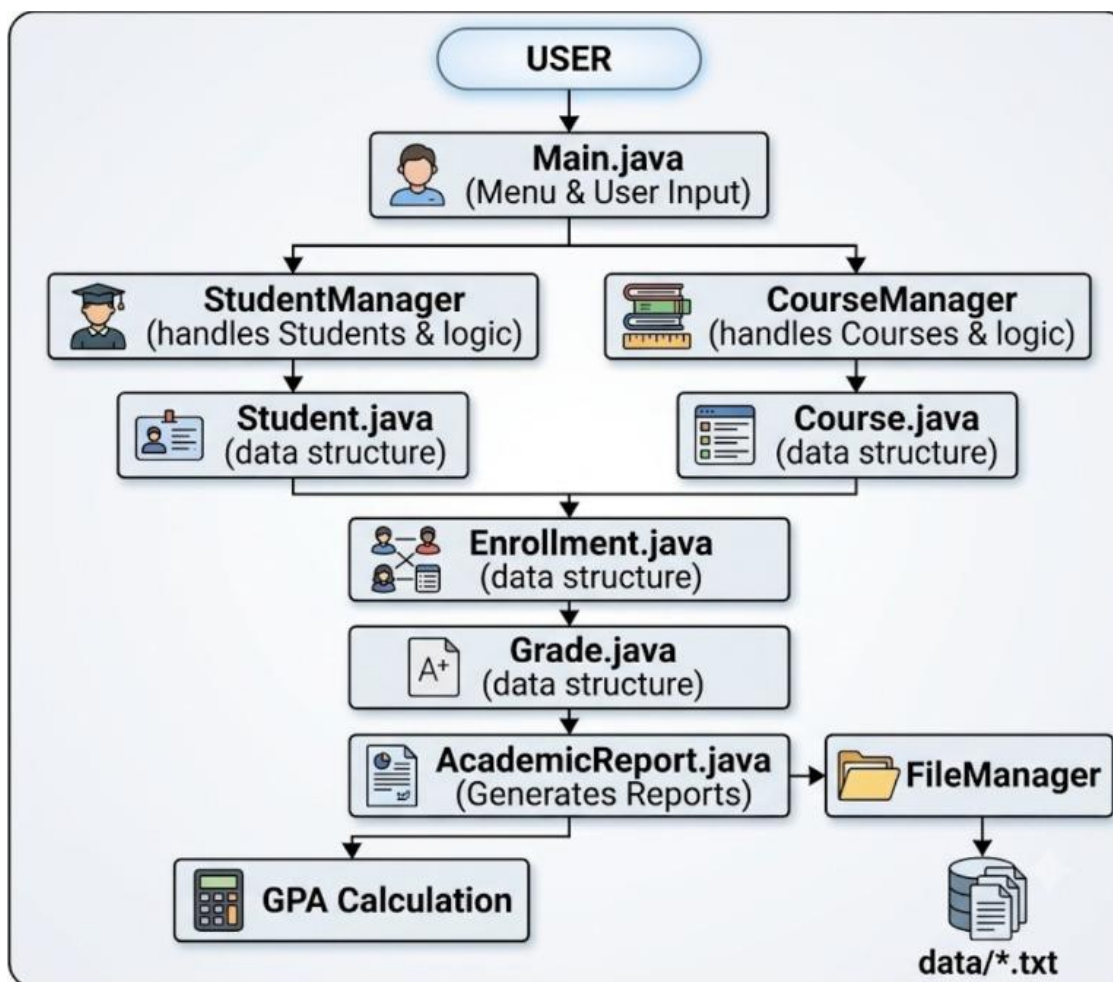
Student records, course catalog, enrollment data, and student grades should be saved in separate text files upon exit or during application execution to persist data between application sessions.

NFR6 – Performance

The application should handle student and academic record management for a small scale efficiently.

5. System Architecture

The project follows a **modular object-oriented architecture**. The `Main.java` class controls the overall program flow and interacts with manager classes, entity classes, the academic report class, and the file storage component.



6. Application Workflow

Student Details



Student Management



Course Details



Course Management



Enrollment



Grade Entry



Academic Report



GPA Calculation



File Storage

Architecture Explanation

Main.java – controls the general execution and collects the inputs from the user.

Student.java – represents the data of student.

StudentManager.java – manages the student objects and provides search of students.

Course.java – represents the course data.

CourseManager.java – manages the courses and provides searching of courses.

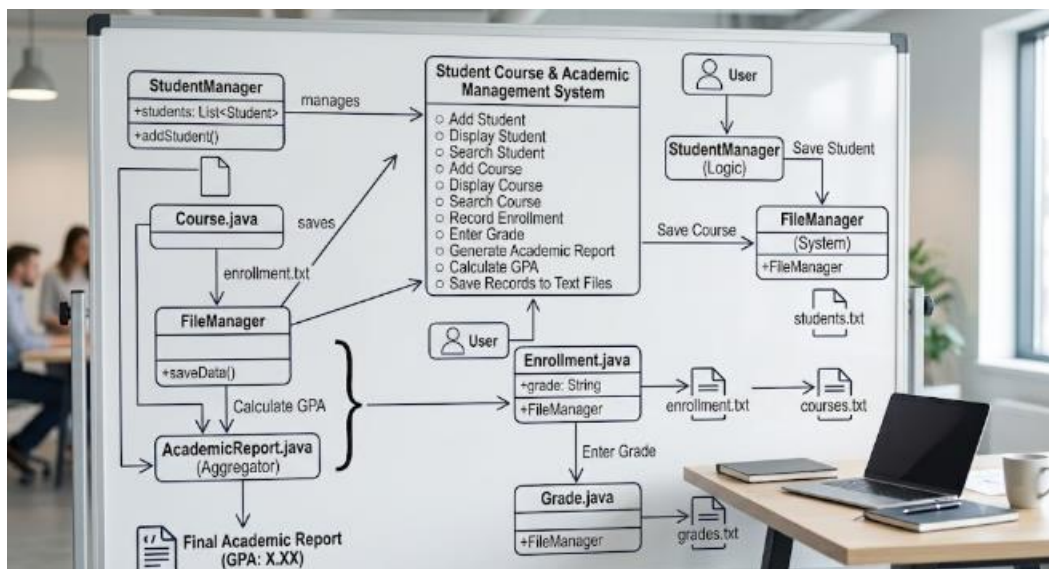
Enrollment.java – relates the student and the course.

Grade.java – retains the grades and computes the grade points.

AcademicReport.java – prepares the academic report and calculates the GPA.

FileManager.java – stores the records in the text files.

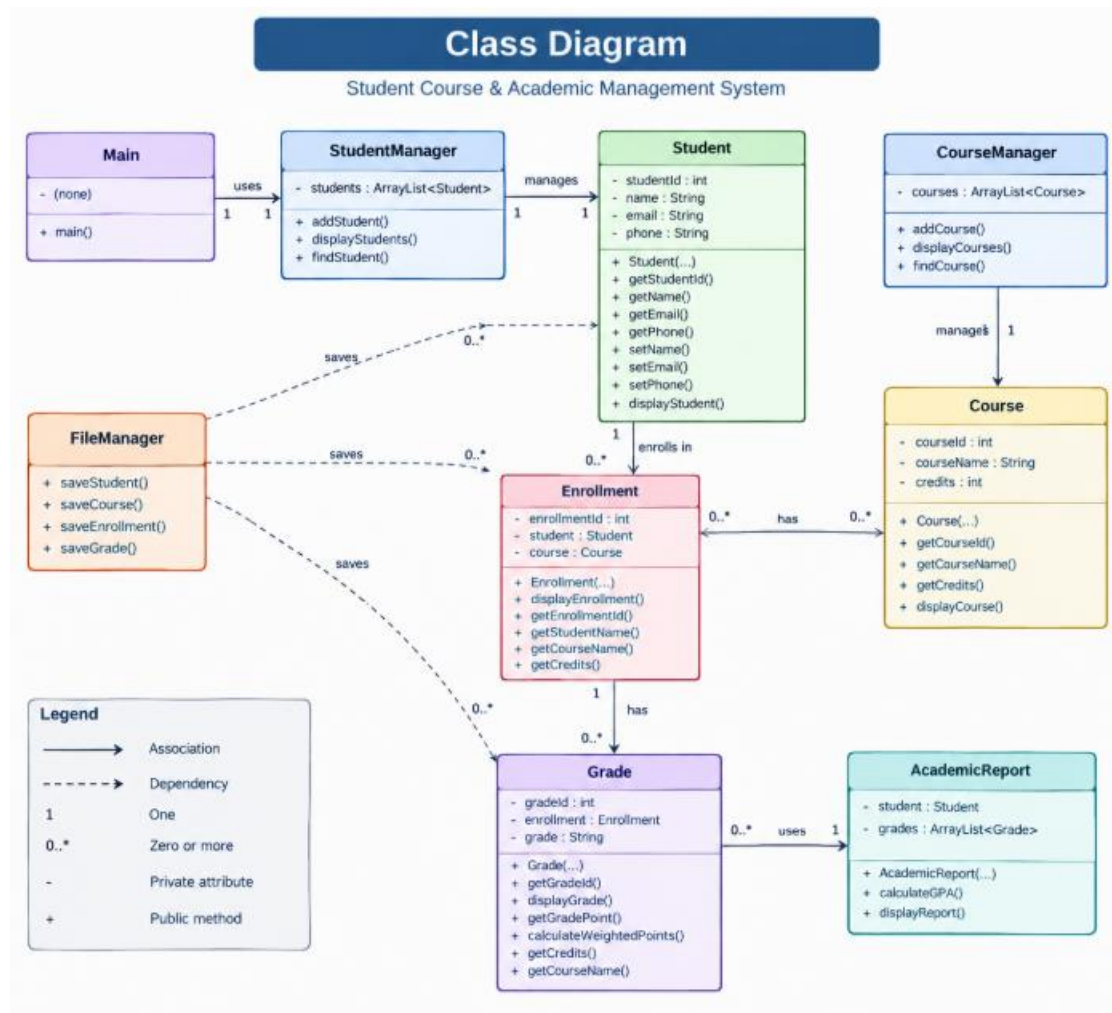
5. Use Case Diagram



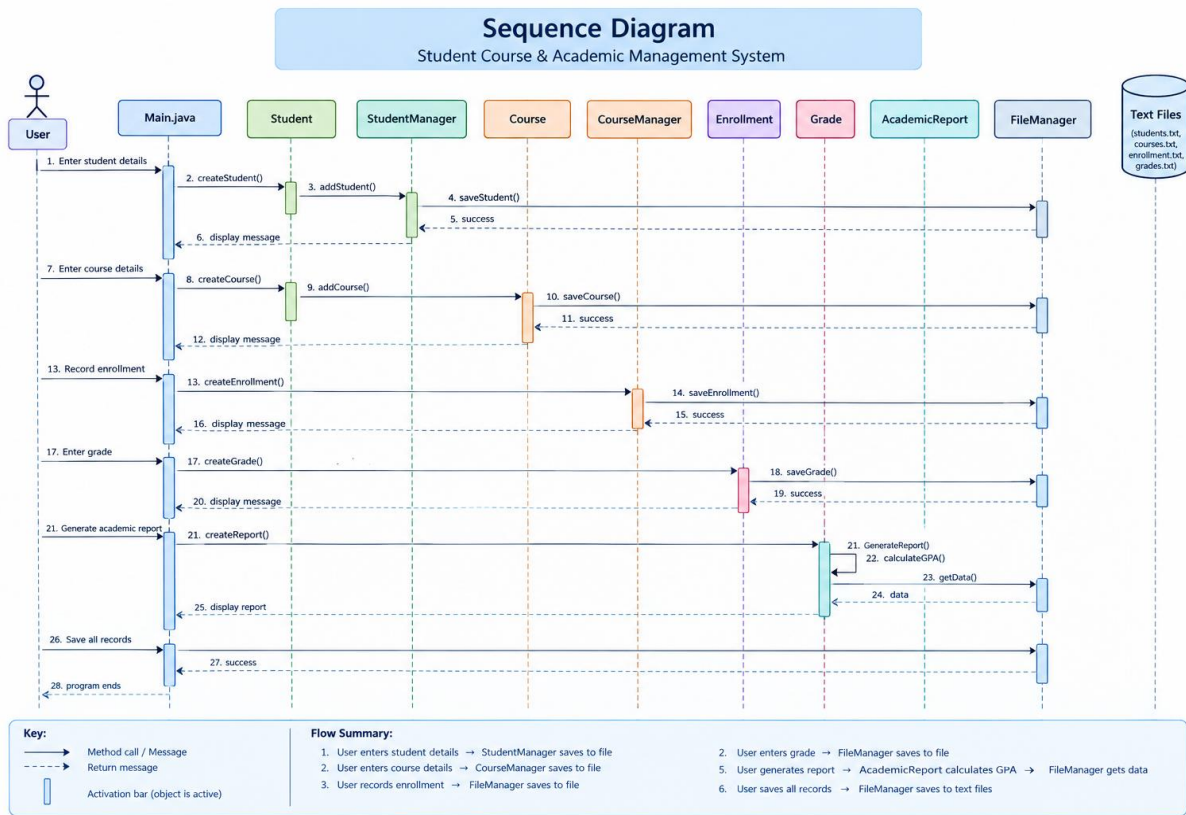
The Use Case Diagram represents the interaction between the user and the Student Course & Academic Management System. The user can manage

student and course information, record enrollments and grades, generate academic reports, calculate GPA, and store records in text files.

6. Class Diagram



Sequence Diagram



7. Project Structure

The project is structured into separate Java classes according to their responsibilities. The folder with the name execution accommodates the java files with sources of the code, and the data folder was used for the text files with the records. Screenshots of the working application were separated in a different folder.

student management system

execution

- Main.java
- Student.java
- Course.java
- Enrollment.java
- Grade.java
- AcademicReport.java
- StudentManager.java
- CourseManager.java
- FileManager.java

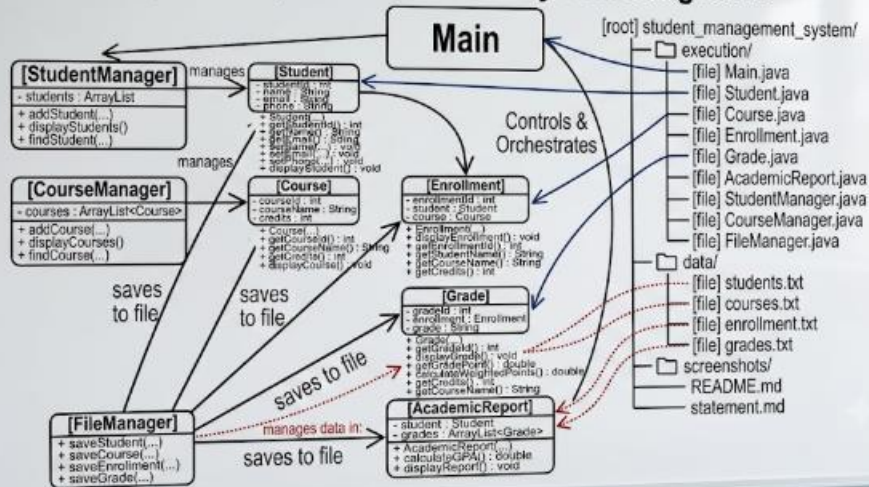
data

- students.txt
- courses.txt
- enrollment.txt
- grades.txt

screenshots

Statement.md

SMS Project Blueprint: Class & File System Integration



8. Implementation

8.1 Student Management

The Student class holds the student's ID, name, email, and phone number while the StudentManager stores student objects and offers the capability to view and search for the student.

8.2 Course Management

We have the Course class that holds the course ID, course name, and credits. Similarly, the Course Manager is tasked with managing course objects, displaying, and searching for a course using the course ID.

8.3 Enrollment Management

The Enrollment class holds the student's data, the course they have enrolled, and the unique enrollment ID. The class offers a feature to display the enrollment's details.

8.4 Grade Management

The Grade class stores the grade of the student's enrollment. The grades are in letter form, for example., A, B, C, D, and E. Besides storing, the class converts the grades into grade points ranging from 4 to 0. We have calculated the weighted points using the formula below.

$\text{Weighted Points} = \text{Grade Point} \times \text{Course Credits}$

8.5 Academic Report

The AcademicReport class takes a student's grade and credits to generate their grades report. In this class, we have implemented the GPA calculation formula as follows:

$\text{GPA} = \text{Total Weighted Points} \div \text{Total Credits}$

8.6 File Handling

The file handling module is managed by the File Manager. It uses the Java FileWriter to write information into files. The files are stored in text format.

The files include:

students.txt

courses.txt

enrollment.txt

grades.txt

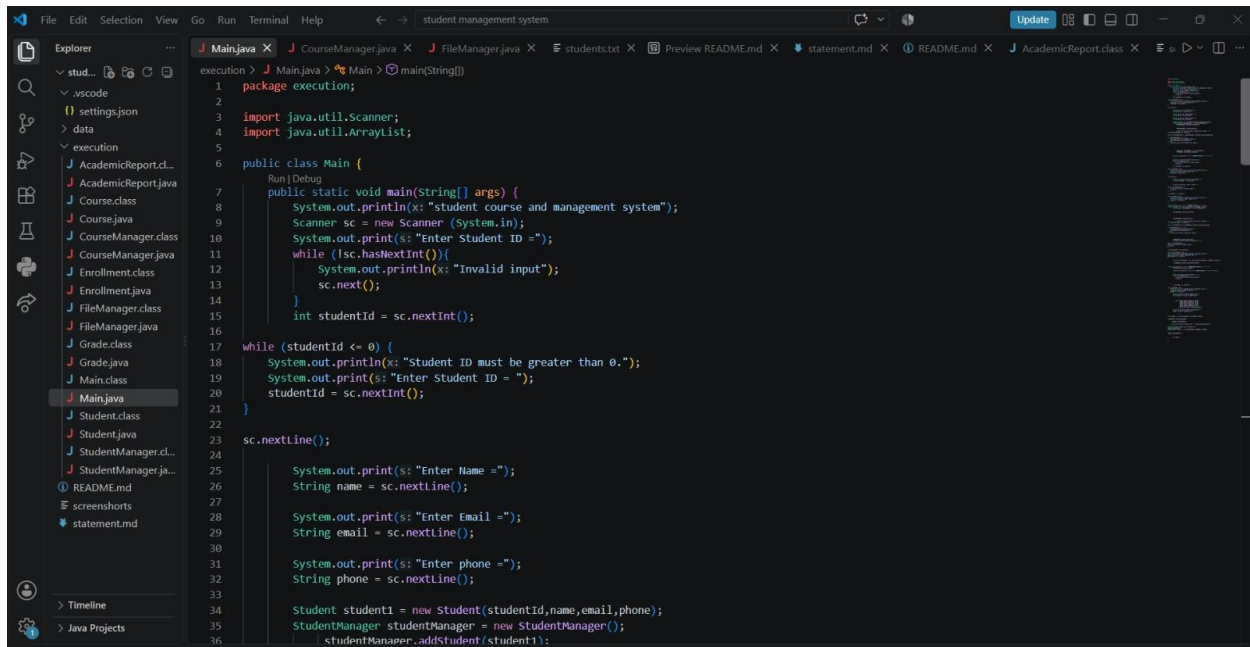
8.7 Input Validation

Finally, the Main class offers the input validation feature for IDs, credits, and grades. When invalid input is detected, the application throws an error. For numeric values, the application only allows decimal numbers. On the other hand, grades are only allowed between A-E.

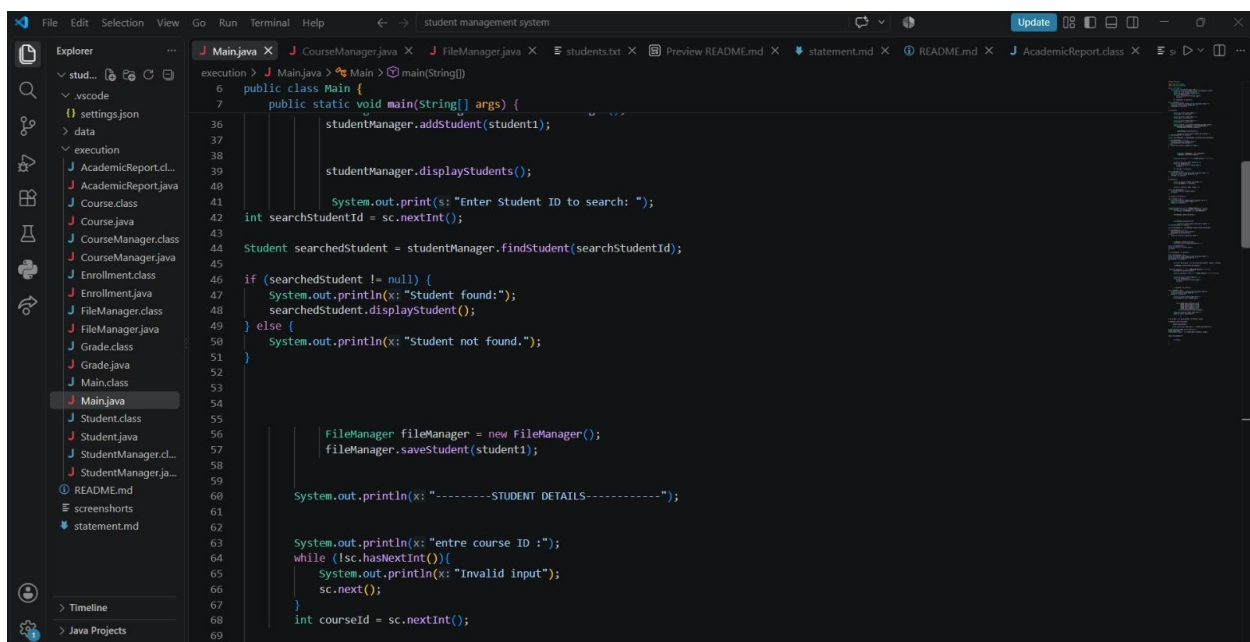
Screenshots & Results

Code-

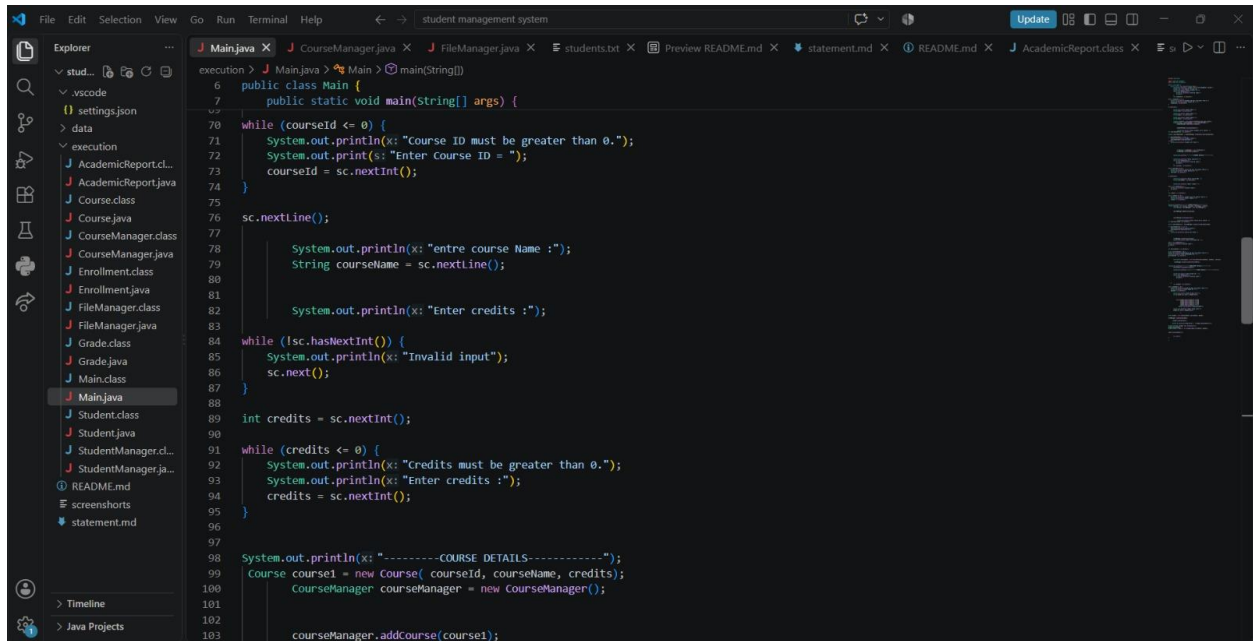
Main.java



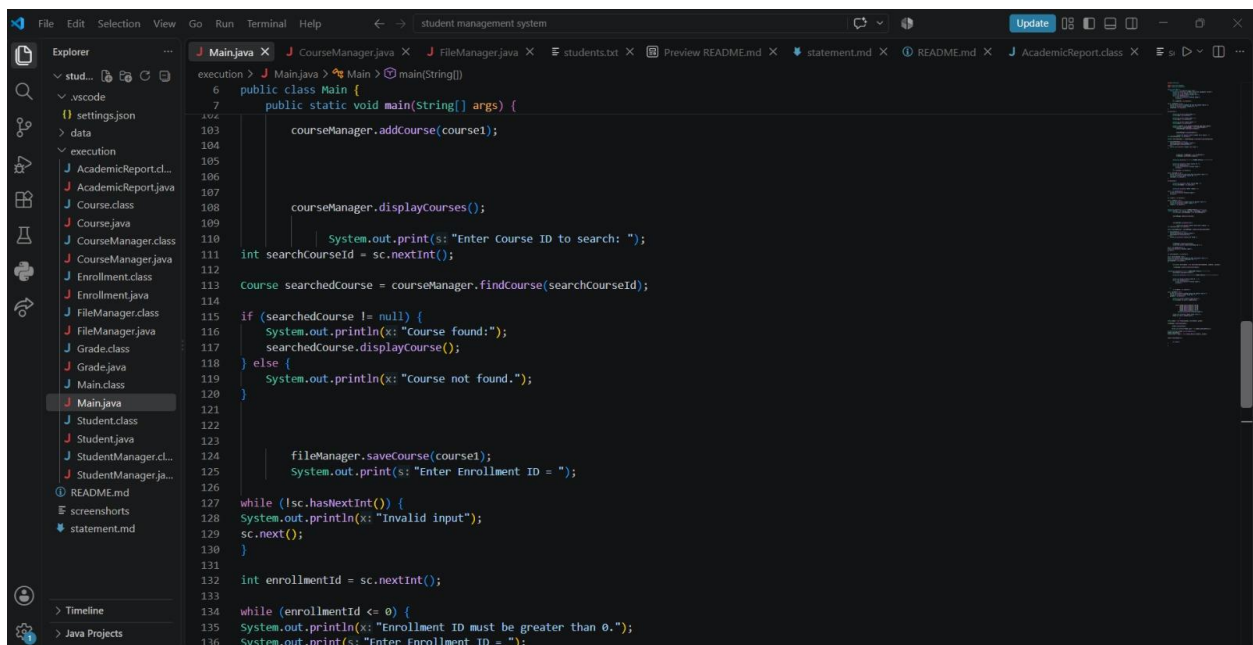
```
1 package execution;
2
3 import java.util.Scanner;
4 import java.util.ArrayList;
5
6 public class Main {
7     Run | Debug
8     public static void main(String[] args) {
9         System.out.println(x: "student course and management system");
10        Scanner sc = new Scanner (System.in);
11        System.out.print(s: "Enter Student ID = ");
12        while (!sc.hasNextInt()){
13            System.out.println(x: "Invalid input");
14            sc.next();
15        }
16        int studentId = sc.nextInt();
17
18        while (studentId <= 0) {
19            System.out.println(x: "Student ID must be greater than 0.");
20            System.out.print(s: "Enter Student ID = ");
21            studentId = sc.nextInt();
22        }
23
24        sc.nextLine();
25
26        System.out.print(s: "Enter Name = ");
27        String name = sc.nextLine();
28
29        System.out.print(s: "Enter Email = ");
30        String email = sc.nextLine();
31
32        System.out.print(s: "Enter phone = ");
33        String phone = sc.nextLine();
34
35        Student student1 = new Student(studentId,name,email,phone);
36        StudentManager studentManager = new StudentManager();
37        studentManager.addStudent(student1);
```



```
38        studentManager.addStudent(student1);
39
40        studentManager.displayStudents();
41
42        System.out.print(s: "Enter Student ID to search: ");
43        int searchStudentId = sc.nextInt();
44
45        Student searchedStudent = studentManager.findStudent(searchStudentId);
46
47        if (searchedStudent != null) {
48            System.out.println(x: "Student found:");
49            searchedStudent.displayStudent();
50        } else {
51            System.out.println(x: "Student not found.");
52        }
53
54
55        FileManager fileManager = new FileManager();
56        fileManager.saveStudent(student1);
57
58        System.out.println(x: "-----STUDENT DETAILS-----");
59
60
61        System.out.println(x: "entre course ID :");
62        while (!sc.hasNextInt()){
63            System.out.println(x: "Invalid input");
64            sc.next();
65        }
66        int courseId = sc.nextInt();
67
68
69
```



```
6 public class Main {
7     public static void main(String[] args) {
8
9         while (courseId <= 0) {
10             System.out.println(x: "Course ID must be greater than 0.");
11             System.out.print(s: "Enter Course ID = ");
12             courseId = sc.nextInt();
13         }
14
15         sc.nextLine();
16
17         System.out.println(x: "entre course Name :");
18         String courseName = sc.nextLine();
19
20         System.out.println(x: "Enter credits :");
21
22         while (!sc.hasNextInt()) {
23             System.out.println(x: "Invalid input");
24             sc.next();
25         }
26
27         int credits = sc.nextInt();
28
29         while (credits <= 0) {
30             System.out.println(x: "Credits must be greater than 0.");
31             System.out.println(x: "Enter credits :");
32             credits = sc.nextInt();
33         }
34
35         System.out.println(x: "-----COURSE DETAILS-----");
36         Course course1 = new Course(courseId, courseName, credits);
37         CourseManager courseManager = new CourseManager();
38
39         courseManager.addCourse(course1);
40     }
41 }
```



```
103         courseManager.addCourse(course1);
104
105         courseManager.displayCourses();
106
107         System.out.print(s: "Enter Course ID to search: ");
108         int searchCourseId = sc.nextInt();
109
110         Course searchedCourse = courseManager.findCourse(searchCourseId);
111
112         if (searchedCourse != null) {
113             System.out.println(x: "Course found:");
114             searchedCourse.displaycourse();
115         } else {
116             System.out.println(x: "Course not found.");
117         }
118
119         FileManager.saveCourse(course1);
120         System.out.print(s: "Enter Enrollment ID = ");
121
122         while (!sc.hasNextInt()) {
123             System.out.println(x: "Invalid input");
124             sc.next();
125         }
126
127         int enrollmentId = sc.nextInt();
128
129         while (enrollmentId <= 0) {
130             System.out.println(x: "Enrollment ID must be greater than 0.");
131             System.out.print(s: "Enter enrollment ID = ");
132         }
133     }
134 }
```

```
1  public class Main {
2      public static void main(String[] args) {
3          Scanner scanner = new Scanner(System.in);
4          System.out.print(s: "Enter Enrollment ID = ");
5          enrollmentId = scanner.nextInt();
6      }
7
8      Enrollment enrollment1 = new Enrollment(enrollmentId, student1, course1);
9      fileManager.saveEnrollment(enrollment1);
10
11      System.out.println(x: "-----ENROLLMENT DETAILS-----");
12      enrollment1.displayEnrollment();
13
14      System.out.println(x: "/////-----GRADE DETAILS-----\\\\\\\\\\\\\\\\");
15
16      System.out.print(s: "Enter Grade ID = ");
17      while (!scanner.hasNextInt()){
18          System.out.println(x: "Invalid input");
19          scanner.next();
20      }
21      int gradeId = scanner.nextInt();
22
23      while (gradeId <= 0) {
24          System.out.println(x: "Grade ID must be greater than 0.");
25          System.out.print(s: "Enter Grade ID = ");
26          gradeId = scanner.nextInt();
27      }
28
29      System.out.print(s: "Enter Grade (A-E):");
30      String grade = scanner.next().toUpperCase();
31  }
```

```
1  public class Main {
2      public static void main(String[] args) {
3          Scanner scanner = new Scanner(System.in);
4          System.out.print(s: "Enter Enrollment ID = ");
5          enrollmentId = scanner.nextInt();
6      }
7
8      Enrollment enrollment1 = new Enrollment(enrollmentId, student1, course1);
9      fileManager.saveEnrollment(enrollment1);
10
11      System.out.println(x: "-----ENROLLMENT DETAILS-----");
12      enrollment1.displayEnrollment();
13
14      System.out.println(x: "/////-----GRADE DETAILS-----\\\\\\\\\\\\\\\\");
15
16      System.out.print(s: "Enter Grade ID = ");
17      while (!scanner.hasNextInt()){
18          System.out.println(x: "Invalid input");
19          scanner.next();
20      }
21      int gradeId = scanner.nextInt();
22
23      while (gradeId <= 0) {
24          System.out.println(x: "Grade ID must be greater than 0.");
25          System.out.print(s: "Enter Grade ID = ");
26          gradeId = scanner.nextInt();
27      }
28
29      System.out.print(s: "Enter Grade (A-E):");
30      String grade = scanner.next().toUpperCase();
31
32      Grade grade1 = new Grade(gradeId, enrollment1, grade);
33      fileManager.saveGrade(grade1);
34      grade1.displayGrade();
35      System.out.println("Grade point : " + grade1.getGradePoint());
36
37      ArrayList<Grade> grades = new ArrayList<>();
38      grades.add(grade1);
39      AcademicReport report = new AcademicReport(student1, grades);
40      report.displayReport();
41      scanner.close();
42  }
```

Output screenshot

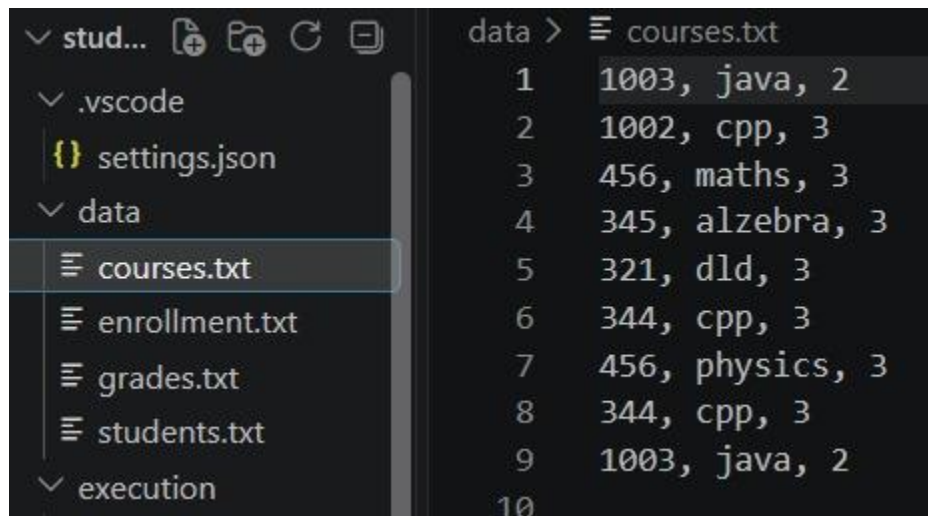
```
Problems Output Terminal ... powershell + v [] [] ... |
PS C:\Users\archita singh\OneDrive\Desktop\student management sy
stem>
PS C:\Users\archita singh\OneDrive\Desktop\student management sy
stem> javac execution\Main.java execution\Student.java execution
\Course.java execution\Enrollment.java execution\Grade.java exec
ution\AcademicReport.java execution\StudentManager.java executio
n\CourseManager.java execution\FileManager.java
PS C:\Users\archita singh\OneDrive\Desktop\student management sy
stem> java execution.Main
student course and management system
Enter Student ID =123
Enter Name =AYUSH
Enter Email =ayush@gmail.com
Enter phone =123456789
Student ID : 123
Name : AYUSH
Email : ayush@gmail.com
Phone : 123456789
-----
Enter Student ID to search: 123
Student found:
Student ID : 123
Name : AYUSH
Email : ayush@gmail.com
Phone : 123456789
-----STUDENT DETAILS-----
entre course ID :
1003
entre course Name :
java
Enter credits :
2
-----COURSE DETAILS-----
Course ID =1003
Course Name =java
Credits =2
-----
Enter Course ID to search: 1200
Course not found.
```

```
-----COURSE DETAILS-----
Course ID =1003
Course Name =java
Credits =2
-----
Enter Course ID to search: 1200
Course not found.
Enter Enrollment ID = 3411
-----ENROLLMENT DETAILS-----
Enrollment ID :3411
Student : AYUSH
Course :java
/////-----GRADE DETAILS-----/////
entre Grade ID = :556
Enter Grade (A-E):A
Grade Id :556
student:AYUSH
course :java
grade :A
Grade point :10.0

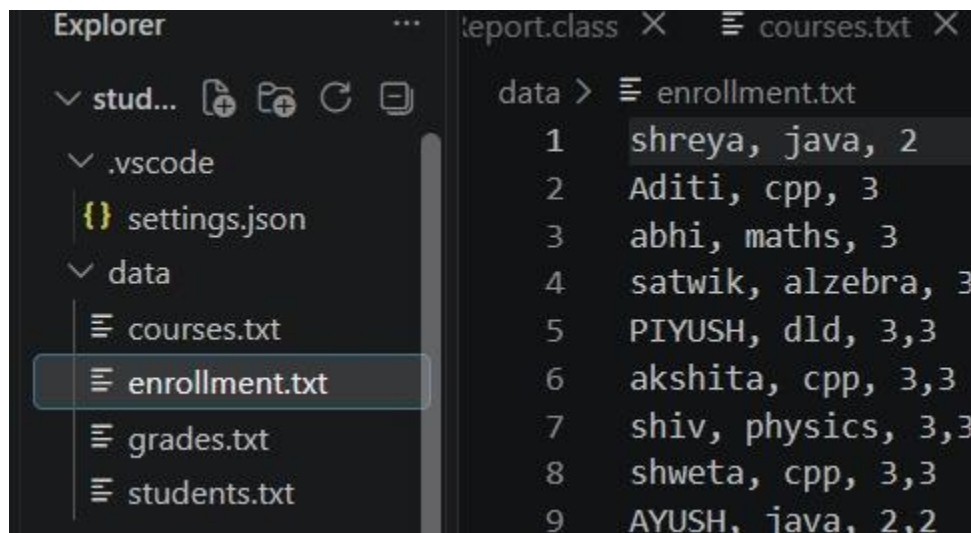
-----ACADEMIC REPORT-----
student ID :123
Name :AYUSH

Course :java
Grade Point: 10.0
Credits : 2
-----
final GPA :10.0
-----
PS C:\Users\archita singh\OneDrive\Desktop\student management sy
stem> 
```

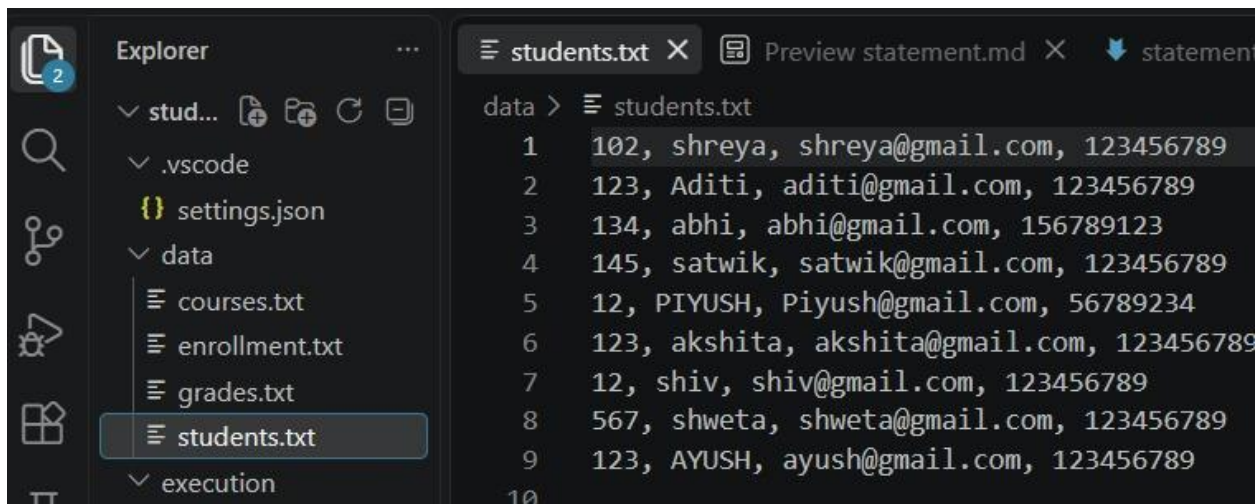
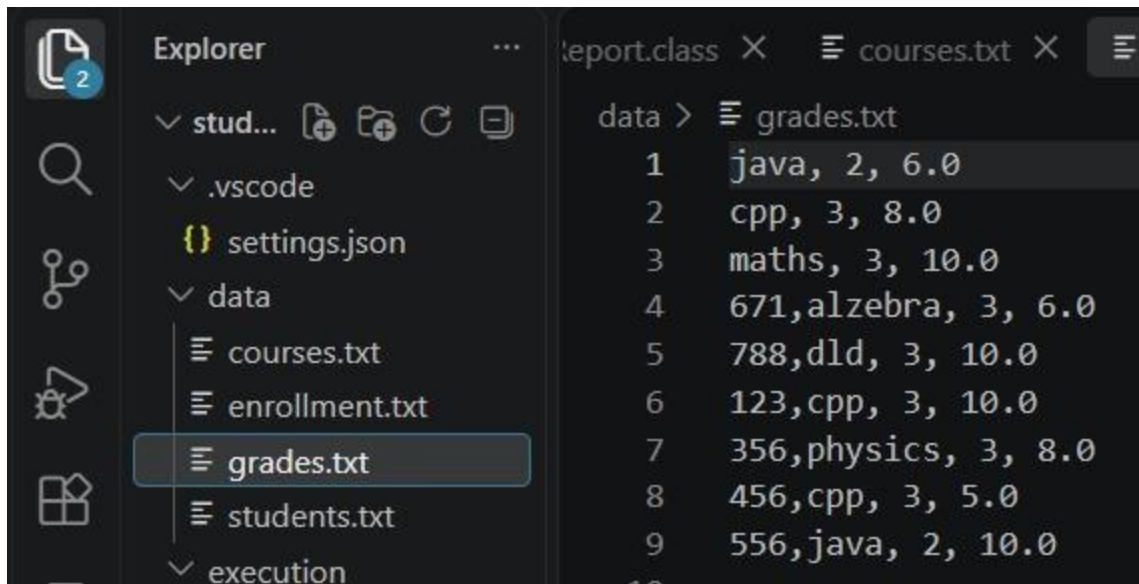
Data storage screenshot



```
data > courses.txt
1 1003, java, 2
2 1002, cpp, 3
3 456, maths, 3
4 345, alzebra, 3
5 321, dld, 3
6 344, cpp, 3
7 456, physics, 3
8 344, cpp, 3
9 1003, java, 2
10
```



```
Explorer
data > enrollment.txt
1 shreya, java, 2
2 Aditi, cpp, 3
3 abhi, maths, 3
4 satwik, alzebra, 3
5 PIYUSH, dld, 3,3
6 akshita, cpp, 3,3
7 shiv, physics, 3,3
8 shweta, cpp, 3,3
9 AYUSH, java, 2,2
```



9. Testing

The application was tested by providing different types of valid and invalid inputs. The main functionalities, including student management, course management, enrollment, grade processing, GPA calculation, and file storage, were tested.

Testing Result

The testing confirmed that the major functionalities of the Student Course & Academic Management System work as expected. Input validation prevents invalid IDs, credits, and grades from being accepted. The system successfully generates the academic report, calculates GPA, and stores records in the required text files.

Challenges

Handling multiple Java classes

Breaking the project into several classes and establishing relationships between them required much attention.

Package and folder structure

Setting up the execution package and executing the java program with an appropriate package structure was initially difficult.

Input Validation

Dealing with invalid numerical inputs and only accepting grades from A to E required additional validation.

File Handling

Storing different record types in different text files requires using Java's file classes and exception handling.

GPA Calculation

Calculating GPA using both grade points and course credits required a weighted-point calculation method.

Learnings & Key Takeaways

I was able to gain much experience through completing this project and applying my Java knowledge to create an application that addresses a real-world issue.

I had the chance to learn Object Oriented Programming with classes and objects.

Practice using ArrayList to store multiple objects.

Learn about encapsulation by applying private access modifiers and the getter and setter methods.

Apply loops, conditions, methods, and constructors.

Acquire new skills and knowledge about how to validate user input.

Learn how to use try-catch blocks to handle exceptions when working with files.

Gain experience by using FileWriter to write information into a file.

Applying my knowledge of creating multiple classes in one project and how they can interact with each other.

I also learned how to properly debug and test my programs. Additionally, I was able to improve my time management and documentation skills. Lastly, I learned how to work with Git and GitHub and push my code to the repository.

Future Enhancements

Add Update/Delete features for students and courses

Add multiple courses and grades for a student

Store data in a database (instead of a text file)

GUI interface for the application

User logins/authentication

Generate report cards for download